

AIQT Milestone 15-RC1 Build Specification v0.2

Implementation Repository Boundary Enforcement

Status: build-ready after v0.2 traceability corrections. This RC hardens M15 source-control discipline after dogfood v4 exposed that an agent can create an implementation app folder while accidentally using a parent repository as the Git context.

1. Source Alignment

This milestone extends AIQT MVP Specification v0.6, Technical Architecture Specification v0.3, and the completed M15 source-control discipline. It preserves the governing pipeline:

```
state -> graph -> packet -> checkpoint -> review/export -> guided workflow -> agent driver ->
management/recovery -> design guidance -> operating discipline -> source-control discipline ->
implementation repository boundary discipline
```

M15-RC1 is not a new feature family. It is a release-candidate hardening patch for M15. It makes the implementation repository boundary explicit before Work Units begin.

2. Milestone Summary

- Name: M15-RC1 - Implementation Repository Boundary Enforcement
- Purpose: ensure the implementation root is its own Git repository root before Work Unit implementation starts.
- Primary failure addressed: agents using a parent/control repository when the generated app folder has no independent Git repo.
- Scope posture: guidance-only. AIQT does not execute Git, verify Git state, create commits, create tags, or call GitHub APIs.

3. Finding Addressed

F063 - Implementation Git repository boundary not enforced before Work Units

- Status: confirmed.
- Severity: high.
- Source: Dogfood v4 GitHub/source-control flow.
- Problem: the agent created a new implementation project folder but used the parent source-control repository instead of initializing Git inside the implementation root.
- Impact: Work Unit commits/tags, rollback, GitHub flow, checkpoint reports, and final review are attached to the wrong repository boundary.
- Resolution: make Git boundary verification a pre-Work-Unit operating precondition in driver, planning, packet, and checkpoint guidance.

4. Non-Goals

- No new public CLI commands.
- No AIQT-executed git commands.
- No automatic repository initialization.
- No automatic commits or tags.
- No GitHub API or gh CLI usage.
- No branch-protection or remote repository automation.
- No checkpoint schema change.
- No schema version bump.

- No direct verification of the reported Git state.

5. Behavioral Position

AIQT guides agents to operate correctly. It does not become a Git automation tool. M15-RC1 strengthens instructions so that source control is anchored to the implementation root itself, not to the AIQT control root or any parent folder.

The implementation root must be the Git repository root. If the user explicitly configures a different branch policy, that policy may override the default main branch expectation; otherwise main is required.

6. Required Repository Boundary Rule

Repository Boundary Rule:

The implementation root must be the Git repository root.
Before modifying implementation files, run:

```
git rev-parse --show-toplevel
```

The returned path must equal the implementation root.

If it returns a parent folder, the AIQT control root, or any path other than the implementation root, stop and correct the repository boundary before continuing.

The agent must not rely on a parent Git repository. The agent must not commit implementation Work Unit changes to the AIQT control repository unless the user explicitly defines the AIQT control root and implementation root as the same repository.

7. Pre-Work-Unit Source-Control Preflight

Before the first implementation Work Unit begins, the agent must self-attest the following from the implementation root:

```
cd <implementation-root>
git rev-parse --show-toplevel
git branch --show-current
git status --short
```

Check	Expected result
Git top-level	Exactly equals the implementation root
Branch	main, unless the user explicitly configured a different branch policy
Status	Clean, or only intentional scaffold files before the baseline commit

If the implementation root is not its own Git repository, the agent must initialize Git inside the implementation root before feature Work Units begin:

```
cd <implementation-root>
git init -b main
# If unsupported:
git init
git branch -M main

# create/update .gitignore before first commit
git add .
git commit -m "chore(repo): initialize implementation repository baseline"
git tag repo-baseline
```

8. Guidance Surface Changes

M15-RC1 must strengthen existing text-only guidance surfaces. It must not add commands.

8.1 Driver Prompt

- Render a source-control boundary preflight before any instruction to start implementation work.
- Explicitly name the AIQT control root and implementation root when known.
- Tell the agent to stop if git rev-parse --show-toplevel does not equal the implementation root.
- Tell the agent to initialize Git inside the implementation root before Work Units if missing.

8.2 Plan Prompt

- When no implementation-root Git repository exists or when repository state is unknown, planning guidance must require a repository initialization/baseline Work Unit before feature Work Units.
- The plan should not treat source control as optional unless the user explicitly disables it.

8.3 Agent Packet

- The Source Control Expectations section must include the Repository Boundary Rule.
- It must instruct agents to run `git rev-parse --show-toplevel` from the implementation root before editing files.
- It must stop agents from committing implementation changes to a parent/control repository.

8.4 Checkpoint Prompt

- Checkpoint guidance must ask the agent to report Git top-level, implementation root, branch, status, commit hash, tag name, risk score, and whether the boundary was verified.
- These fields remain self-attested formatted text inside existing checkpoint summary/notes fields.

9. Self-Attestation and Verification Boundary

All Git boundary reporting is self-attested by the agent. AIQT does not execute Git commands and does not verify the returned paths, branch names, commits, or tags.

Required checkpoint source-control report fields:

- Implementation root
- Git top-level from `git rev-parse --show-toplevel`
- Boundary verified: yes/no
- Current branch
- Git status before checkpoint
- Commit created: yes/no
- Commit hash if created
- Tag created: yes/no
- Tag name if created
- Uncommitted files remaining
- Ignored/generated artifacts
- Risk score 0-100
- Mapped AIQT severity: low | medium | high | critical
- Risk rationale

10. Packet Audit Metadata

M15-RC1 may reuse the existing M15 includesSourceControlGuidance audit flag. If a separate flag is useful, it must be additive and optional. The authoritative historical audit record remains the `agent_packet.created` runlog event data. `state.lastAgentPacket` remains only a latest-packet mirror.

No new runlog event type is required. No full packet body persistence is required.

11. Repository Implementation Scope

Implementation should modify existing source-control discipline and prompt/packet rendering files wherever they currently live in the repository. Do not create new schema files solely to satisfy this specification. Prefer extending the existing M15 source-control discipline helper and existing M14/M15 audit metadata paths.

Likely affected areas:

- source-control discipline rendering helper
- driver prompt template
- plan prompt template
- checkpoint prompt template
- agent packet template/service
- agent packet audit metadata tests if wording or flags change
- export template only if a new explicit boundary audit line is added

12. Test Scope

Add focused tests only. This is an RC patch.

Required unit tests:

- boundary rule text includes git rev-parse --show-toplevel
- expected top-level equals implementation root
- parent Git repository usage is explicitly blocked
- branch expectation preserves main unless configured branch policy

Required integration/regression tests:

- driver prompt includes repository boundary preflight
- plan prompt requires repository initialization/baseline guidance when Git repo is missing or unknown
- agent packet Source Control Expectations includes boundary rule
- checkpoint prompt asks for Git top-level and boundary verified fields
- no new commands are registered
- no exit-code contract changes
- no new runlog event types are introduced

13. Validation

Primary validation command:

```
pnpm validate
```

Fallback if blocked by the known ERR_PNPM_IGNORED_BUILDS/esbuild gate:

```
node_modules/.bin/tsc --noEmit
node_modules/.bin/eslint .
node_modules/.bin/vitest run
node_modules/.bin/tsc
```

If pnpm validate creates lockfile/workspace side effects in this environment, revert unintended side effects before staging.

14. Definition of Done

- F063 is referenced consistently and no finding number collision remains.
- The full source-alignment pipeline is present and includes source-control and implementation-repository-boundary discipline.
- Driver, plan, packet, and checkpoint guidance all include implementation-root Git boundary checks.
- Guidance explicitly says the implementation root must be the Git repository root.
- Guidance explicitly says parent/control Git repository usage is a boundary error.
- Branch expectation is main unless user configured a different branch policy.

- All reporting remains self-attested in existing checkpoint fields.
- No new commands, Git execution, GitHub API calls, automatic commits, automatic tags, or checkpoint schema changes are introduced.
- Tests pass.
- Working tree is clean after commit.

15. Required Implementation Report

- Branch name
- Commit hash
- Files changed summary
- Whether F063 is used consistently
- How driver/plan/packet/checkpoint guidance changed
- Whether packet audit metadata changed or reused existing includesSourceControlGuidance
- Schema changed? If yes, explain why additive/optional only
- AIQT_SCHEMA_VERSION changed? Expected: no
- Commands changed? Expected: no
- Runlog event types changed? Expected: no
- Validation results
- Final test count
- Deviations from spec
- Deferred/unresolved items
- Working tree status after commit

16. Dogfood v4 Continuation Gate

Before resuming dogfood v4, the agent must correct implementation repository ownership:

```
cd <implementation-root>
git rev-parse --show-toplevel
# returned path must equal <implementation-root>

# if not, stop and initialize a dedicated repository in <implementation-root> before any more Work
Units.
```

Dogfood v4 should not continue while the implementation app is using the parent/control repository as its Git top-level.